Data Quality Report

Dataset: 17 tables, roughly 440,000 event records, 1 January to 8 August 2026. Analysis window: January to July 2026, seven complete months. Everything below rebuilds from the raw files with `python src/run_sql.py`.

What we found

We investigated eight data quality issues. Four were confirmed, three were rejected after testing, and one is not on the list the brief gave us.

The rejections matter as much as the confirmations. If we had acted on the apparent duplicate-reference problem without testing it, we would have deleted ₹20.54 Cr of legitimate recovery, which is nearly eight times larger than the one duplication problem that turns out to be real.

Issue	Verdict	Impact
Duplicate payment records	Confirmed	₹2.59 Cr removed, 1.93%
Duplicate payment references	Rejected	Would have wrongly removed ₹20.54 Cr
Agent identity corruption	Confirmed	Three required dimensions lost
Borrower record conflicts	Confirmed	64% of borrower rows rejected
Timezone inconsistency	Immaterial	Calling-time analysis void
Disposition code drift	Rejected	None
Denominator attrition	Rejected	None
Contact-grain mismatch	Confirmed, not in the brief	RPC not computable across tables

After all cleaning, reported SUCCESS recovery falls from ₹134.15 Cr to ₹131.56 Cr across the full data period, and to **₹126.85 Cr** inside the January to July window.

1. Duplicate payment records — confirmed

A duplicate payment can arise three different ways, and each needs a different fix, so we tested all three separately. The same payment_id means an ingestion double-load. The same payment_reference with a different payment_id means a gateway retry. No shared identifier at all, but the same account and amount seconds apart, means an application double-submit.

The first level is real: **500 excess rows**. Of those, 486 are exact full-row duplicates and 14 differ only in payment_reference, where one copy has it and the other is null.

The third level came back empty at every window we tried, from 10 seconds out to an hour. There is no double-submit problem.

We deduplicate on payment_id and keep the row that has a reference populated, since that is the more complete record.

The impact is **₹2.59 Cr of reported recovery, 1.93% of the total**. What matters more than the size is the shape: the inflation sits between 1.50% and 2.27% in every single month. Duplicates raise the level of reported recovery but create no trend, so they cannot explain a claimed month-on-month improvement.

2. Duplicate payment references — rejected

This one looked like the bigger problem. payment_reference has 4,678 duplicated values against 500 duplicated IDs, roughly nine times as many, worth ₹20.54 Cr.

Rather than deduplicate on it, we asked what would have to be true if these were genuine gateway retries. A retry is the same borrower paying the same amount twice, so within a reference group the account and the amount should match.

They never do. In **all 3,407 collision groups, both account_id and amount differ**. Not one of them looks like a retry.

We then checked whether the collisions are simply chance. References run from TXN0000009 to TXN0069996, a pool of 69,996 values. Drawing 24,632 references at random from that pool predicts 20,765 distinct values. We observe 20,821, a difference of 0.3%.

So payment_reference is not an identity key at all. It is a randomly assigned label that collides at exactly the rate the birthday paradox predicts.

Deduplicating on it would have removed 3,811 legitimate payments worth ₹20.54 Cr and understated true recovery by 15%. This is the largest error we avoided, and we only avoided it by trying to disprove the finding rather than acting on it.

3. Agent identity corruption — confirmed

The brief asks whether the same agent shows up under several identifiers. The real problem is worse than that and needed a different test.

agents.csv has 30,000 rows for 1,000 distinct agent_id values. If this were a valid slowly-changing dimension, meaning an append-only history of changes to each agent's record, then joined_at would have to be immutable. A person's hire date does not change when their team changes.

It is not immutable anywhere. **All 1,000 agent IDs carry conflicting hire dates**, with one distinct value per row, and **all 1,099 employee codes span more than one agent ID**. Each agent ID also carries 6 to 10 different names and 7 to 15 different vendors. The relationship is many-to-many in both directions.

Every row having a distinct hire date is the giveaway. Even a badly maintained real system would repeat some values.

We reject the table as a dimension and keep agent_id as an opaque key only. It has zero orphans across the six tables that reference it, so per-agent aggregation is still valid. Agent attributes are not.

This costs us three of the thirteen dimensions the brief asks for: agent tenure, agent team, and vendor attributed through the agent. We report that rather than working around it, because any reconstruction

would be arbitrary. Vendor is recovered by a different route, through calls.vendor_id and payments.provider_id, which both resolve cleanly.

4. Borrower record conflicts — confirmed

borrowers.csv has 30,600 rows for 11,015 distinct IDs, including 600 exact duplicates. **8,159 borrower IDs, 74% of them, have a conflicting city**, and 8,185 have a conflicting name.

While checking that, we tested referential integrity on every foreign key in all 17 tables and found something more serious. account_id has zero orphans across all twelve tables that reference it. borrower_id has between 761 and 985 orphan IDs in every one of the nine tables that reference it, affecting up to 9,778 rows. On top of that, 455 accounts have a null borrower_id and 897 account-level borrower IDs point at borrowers that do not exist.

Two things follow. First, account_id is our analytical spine. That is an evidence-based decision, not a convention. Second, where we cannot avoid borrower attributes, such as geography, we take the most recently updated row.

The borrower table loses **64% of its rows in cleaning**, from 30,600 down to 11,015, which is the largest single cleaning impact in the pipeline. Any borrower-level metric carries a 7 to 8% orphan rate, and geography results inherit both that and our tiebreak rule. We report geography with that caveat attached.

5. Timezone inconsistency — immaterial

Three timezone labels appear on calls in near-equal proportion: UTC, Asia/Kolkata and Asia/Dubai, at roughly 30,400 rows each. They also appear on accounts, agent_sessions and vendor_telephony, where they disagree row by row. Kolkata is UTC+5:30 and Dubai is UTC+4:00, so the same wall-clock string means three different moments.

There were two possibilities with opposite fixes. Either the timestamps are already local, in which case we must convert before comparing hours, or they are all UTC with the label as metadata, in which case no conversion is needed but any reported local hour is wrong by up to five and a half hours.

Human calling has a strong daily rhythm, which gives a natural test. If the three groups peak at the same hour they share a clock. If they peak at offsets of 0, +5:30 and +4:00, the timestamps are genuinely local.

Neither happened. All three distributions are flat, between 3.92% and 4.58% against a uniform expectation of 4.17%. Day of week is flat too, 13.98% to 14.68% against 14.29%, with Saturday the busiest day.

So we apply no timezone conversion. With no daily or weekly signal there is nothing to align against, and correction is both impossible and pointless.

The consequence is that **calling time, one of the thirteen required dimensions, cannot be analysed**. That is worth stating as a finding rather than an absence. Anyone who regresses contact rate on hour of day in this data will find noise, and might well report it as a result.

6. Disposition code drift — rejected

call_dispositions carries nine codes across three version labels, legacy, v1 and v2, and it contains both PTP and PROMISE_TO_PAY as separate codes with almost identical counts. If a system had migrated from one

to the other partway through the year, a report counting only one of them would show a step change with no change in behaviour behind it.

Two conditions have to hold for that trap to be live. Version has to correlate with time, and the code vocabulary has to differ between versions.

Neither holds. Version share is roughly 33% each in every month, so no migration happened. And all nine codes appear in all three versions at roughly 1,300 each, so the version labels are attached to nothing.

There is still a definitional trap, though it is not a drift problem. Counting only PTP gives a rate around 11.2%; counting both codes gives around 22.4%. **The choice halves the metric**, by a constant 10.7 to 11.8 percentage points every month. We count both.

7. Denominator attrition — rejected

If accounts that fail to pay get closed or written off, later months end up computed on a pool of survivors and conversion rises with no change in behaviour. We ran three checks.

Exit statuses account for between 42.07% and 43.90% of monthly status transitions, which is flat. The cumulative count of accounts that have ever exited is linear at around 2,400 a month with no steepening. And distinct accounts targeted per month sits between 5,160 and 5,800, with the SKIPPED share holding between 24.0% and 25.7%.

Nobody is disappearing from the population. Rejected.

Denominator choice, however, turns out to be the single largest distortion in the whole analysis, even without anyone manipulating it. The same numerator against three different denominators tells three different stories. Per worked account moves between −1.6% and +1.8%. Per targeted account moves between −3.4% and +2.2%. **Per total book, using a fixed 30,000, moves between −8.5% and +11.3%.**

The reason is that worked and targeted denominators move with activity, so in a short month the numerator and denominator fall together and the rate holds steady. A fixed denominator absorbs nothing, so every fluctuation passes straight through and gets amplified. The reported 11% corresponds exactly to that third series.

8. Contact-grain mismatch — confirmed, and not on the brief's list

We found this while computing RPC, which came back above 100%. That is impossible, so we reconciled the three call tables.

All call IDs resolve cleanly, so this is not an orphan problem. The tables simply encode three incompatible definitions of having reached someone. Counting calls with a CONNECTED attempt gives 21,109. Counting calls marked ANSWERED gives 17,896. Counting calls with any disposition record gives 28,971. These do not nest and do not agree, and the spread between them reaches 62%.

The decisive contradiction is this: **only 8,056 of 35,000 dispositions, 23% of them, sit on a call that had a CONNECTED attempt.** The other 77%, including PROMISE_TO_PAY, PAID and DISPUTE, are recorded against calls that never connected. An agent cannot take a promise to pay on a call that did not connect.

We now compute RPC entirely inside call_dispositions, which gives 65.51% to 66.84%, and contact rate entirely inside call_attempts, which gives 19.66% to 20.56%. The two are reported separately and are explicitly not reconcilable.

The business consequence is worth naming. **Any contact rate or RPC figure reported today depends on which table the query happened to hit**, and there is no way to tell from the output which definition was used. That is an operational reporting risk independent of the 11% claim.

Other things worth noting

Null rates are uniformly around 1 to 3% and suspiciously round. calls.agent_id is exactly 2.00% null, call_attempts.vendor_id is exactly 2.00%, and field_visits.scheduled_at is exactly 1.00%. Real missingness is never that tidy. We handle missing values by excluding them from the relevant denominator rather than imputing, since imputing would fabricate activity that did not happen.

accounts.dpd is a static snapshot, one value per account with no time dimension, and account_status_history carries status but not DPD. So DPD has to be treated as a fixed account attribute. That introduces its own bias, since we do not know when it was measured, and we state it as an assumption.

Late-arriving and out-of-window events are negligible. Five rows in calls fall outside 1 January to 8 August, one in December 2025 and four between 9 and 12 August, out of 440,000 events. The README lists conflicting timestamps and late-arriving events as injected defects; their measured footprint is 0.001% of rows.

Four dimensions the brief asks for have no source column at all: language, client, agent tenure beyond the corrupted joined_at, and any cost, spend or rate-card field. The missing cost data means **cost per rupee recovered, ROI and break-even cannot be computed from this dataset**, which matters directly for the ₹10 Cr question.

Cleaning waterfall

Produced by `clean.waterfall` and recomputed on every pipeline run.

Table	Raw	Kept	Rejected	Rule
accounts	30,000	30,000	0	D1 spine
borrowers	30,600	11,015	19,585	D14 latest updated_at
payments (dedup)	25,500	25,000	500	D3/D5
payments (window)	25,000	24,086	914	D2 Jan–Jul
payments (SUCCESS)	24,086	16,918	7,168	Excl. FAILED/PENDING/REVERSED
calls	91,350	86,867	4,483	dedup + D2
call_attempts	120,000	115,627	4,373	dedup + D2
call_dispositions	35,000	33,782	1,218	dedup + D2
whatsapp_events	60,600	57,817	2,783	dedup + D2
sms_events	45,000	43,395	1,605	dedup + D2

Table	Raw	Kept	Rejected	Rule
field_visits	25,000	24,129	871	dedup + D2
daily_targeting	45,000	43,399	1,601	dedup + D2
promises_to_pay	18,000	17,315	685	dedup + D2

The golden dataset comes out at 210,000 rows, being 30,000 accounts across 7 months, holding ₹126.85 Cr of recovery.

Build-time contracts

Seven assertions run on every build, in `golden.contract_checks`. A failure stops the build rather than logging a warning.

Check	Status	Observed
Row count is 210,000	PASS	210,000
Primary key unique	PASS	0 violations
All 30,000 accounts present each month	PASS	30,000
Seven months present	PASS	7
No negative amounts	PASS	0
No null account attributes	PASS	0
Recovery reconciles to cleaned source	PASS	₹126.85 Cr

The last one is the most useful. It proves the golden layer neither lost nor invented money relative to the cleaned payment records. Every pipeline should have at least one end-to-end value reconciliation, not only structural checks.

A note on the dataset README

The dataset ships with a README listing twelve intentionally injected defect classes. We tested each one rather than accepting it.

Two of the twelve do not survive measurement. The duplicate payment references are indistinguishable from random collision, as shown above, and the conflicting timestamps and late-arriving events amount to five rows in 440,000.

Had we treated that README as ground truth, the reference deduplication alone would have removed ₹20.54 Cr of legitimate recovery. Documentation describing what should be wrong with a dataset is a hypothesis to test, not a finding to report.